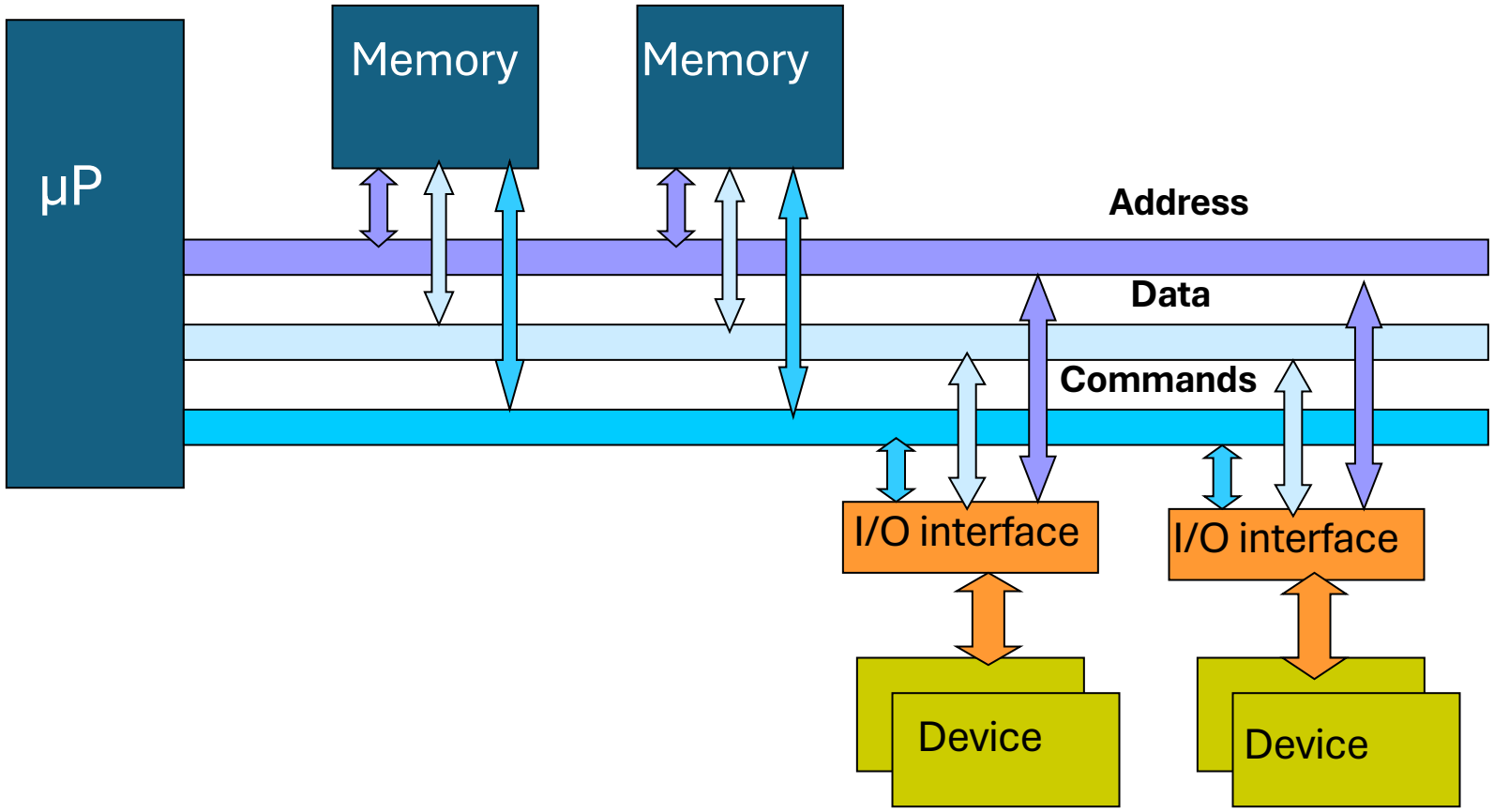
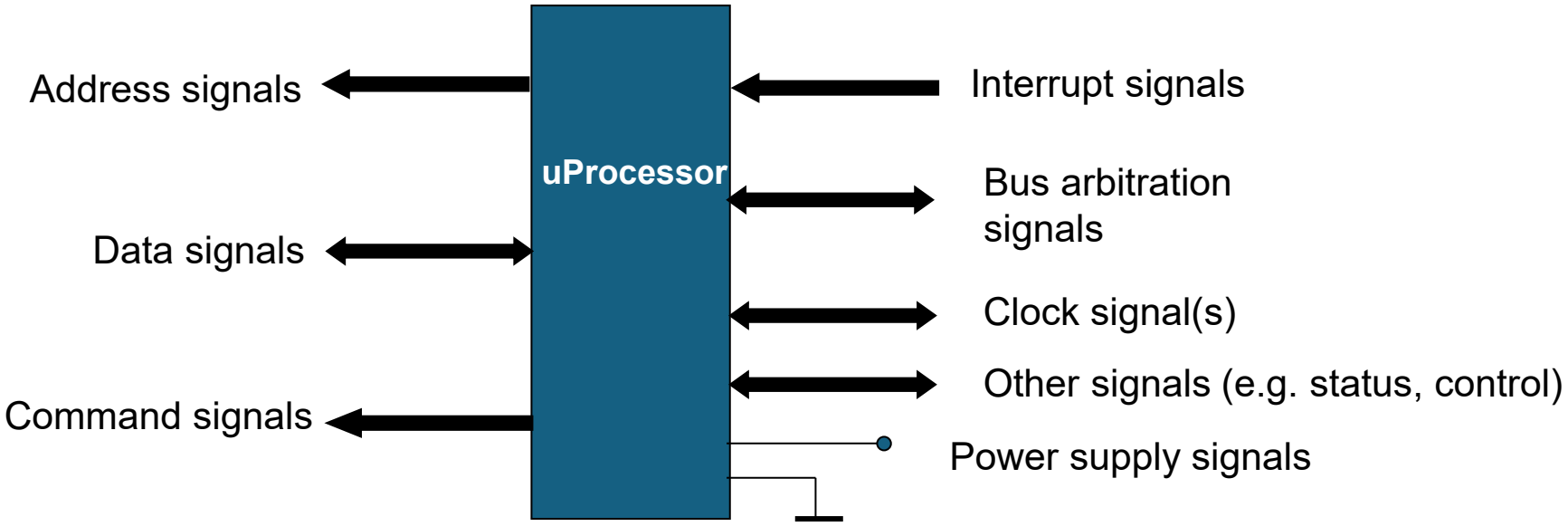
Computer architecture

review

Computer system structure



Microprocessor signals



Microprocessor signals

- Address signals: A_0-A_n
  - Used to specify memory locations or I/O ports (registers)
  - Generated by the microprocessor towards other components in order to access them (read or write operations)
  - The number of address lines determines the maximum address space of a microprocessor
    - 20 lines => 1MB
    - 32 lines => 4GB
- Data signals: D_0-D_m
  - Bidirectional lines used to transfer instruction codes and data between the microprocessor and the other components of the system
  - The number of data lines usually matches the internal organization of the processor (there are also exceptions, see 8088, Pentium Pro)
  - The number of data lines determines the maximum width of the data transferred on a bus
    - E.g.: 8, 16, 32, 64 lines

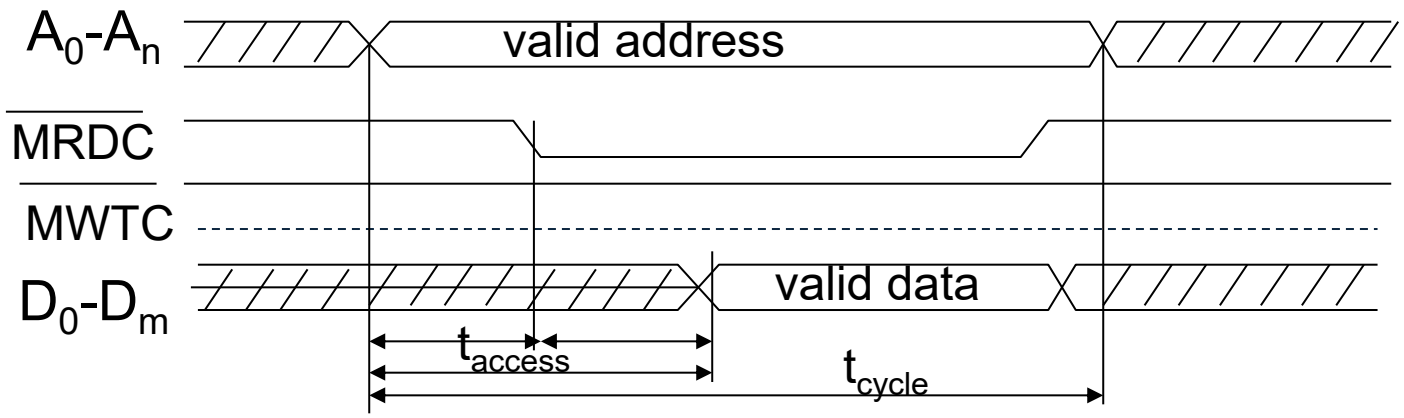
Typical microprocessor signals

- Command signals:
  - MRDC\, MWTC\, IORC\, IOW\, INTA\
  - determine read/write cycles, for memory or interface
  - Similar signals for different types of processors
- Control signals: ALE (Address Latch Enable), DEN (Data Enable)
  - Control the address and data buffers
  - Specific to each type of processor
- Interrupt signals: INTR, NMI
- Clock signals: CLK, PCLK
- Power supply signals: GND, +5V, 3.3V

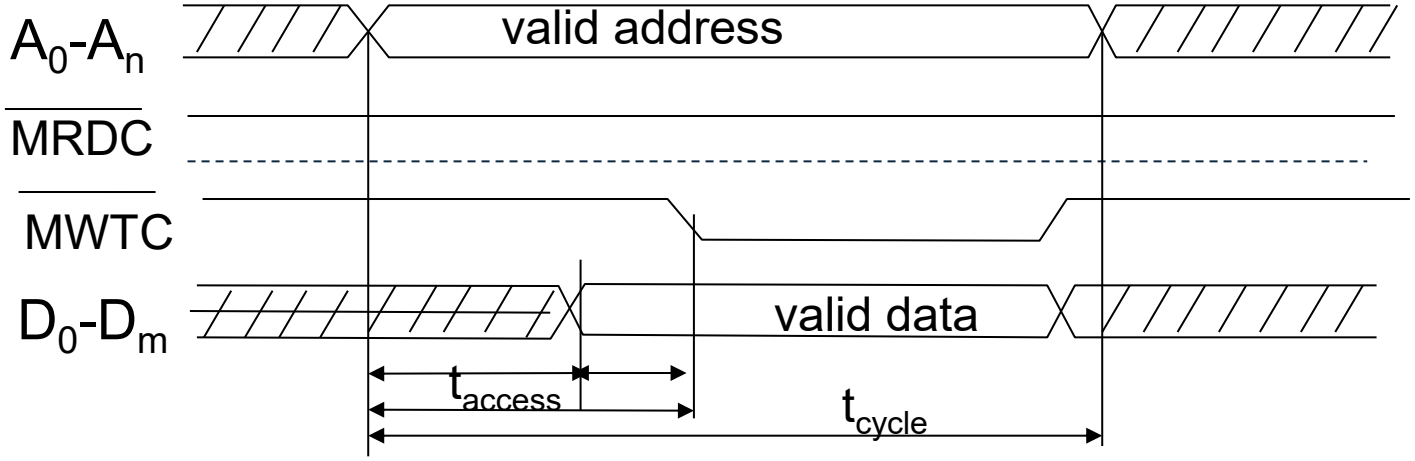
Instruction execution

- Inside: read instruction, decode, execute, memory op., write back
- Outside: instruction read cycle, data read/write cycle (data from memory or I/O)

Memory read



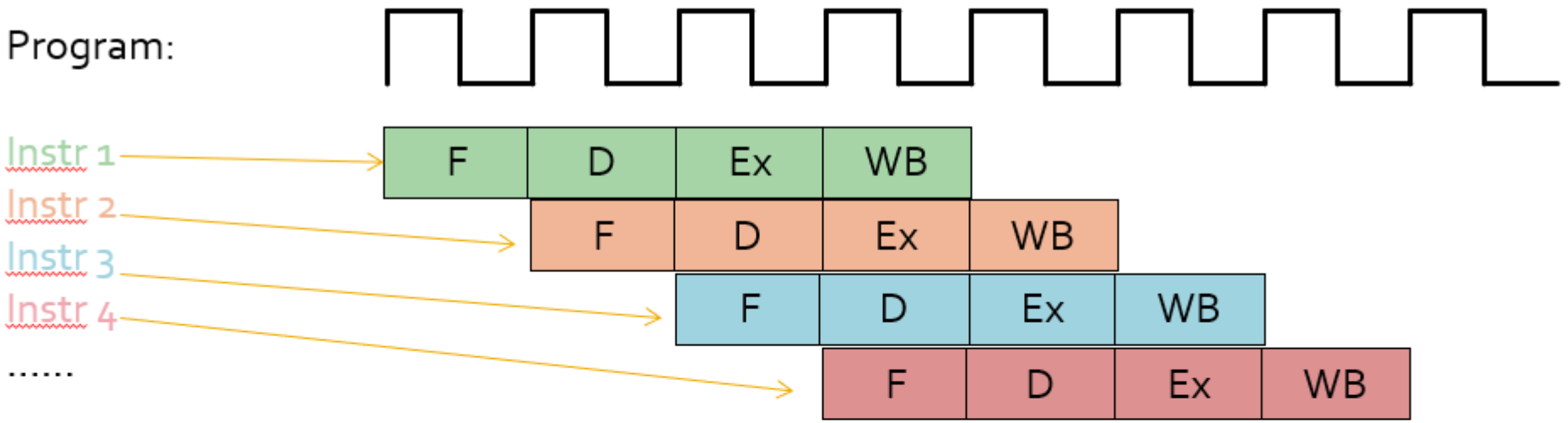
Memory write



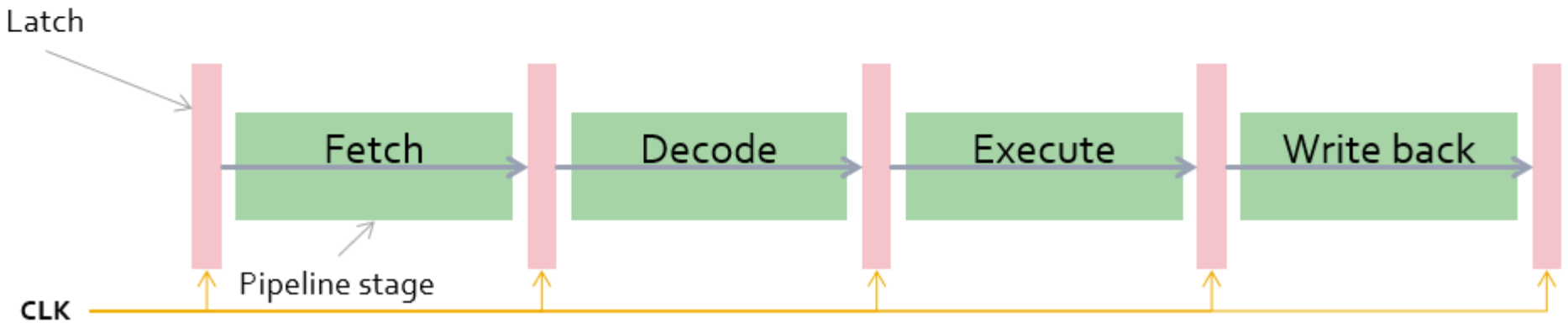
Technologies to enhance performance

- Multiple levels of cache memories (L1 Data/Instruction,L2, …)
- Pipelined execution
- Superscalar architectures
- Memory pre-fetch
- Branch prediction, speculative execution
- Out-of-order execution
- HW support for multi-threading
- Multi-core architectures
- SIMD execution

Pipeline



CPI=1? Hazard situations!



Hazards

- Data (data dependencies)
- Structural (structural dependencies)
- Control (branches)

Data dependency

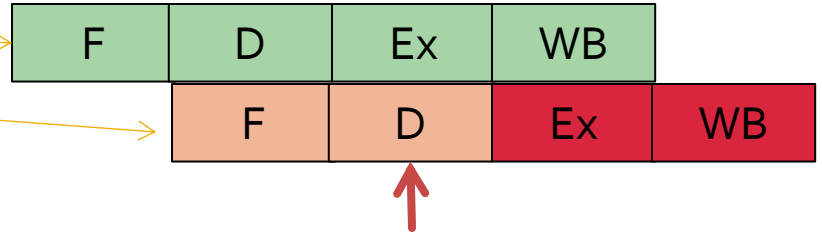
Program:

....

A=B*2

C=A+1

....



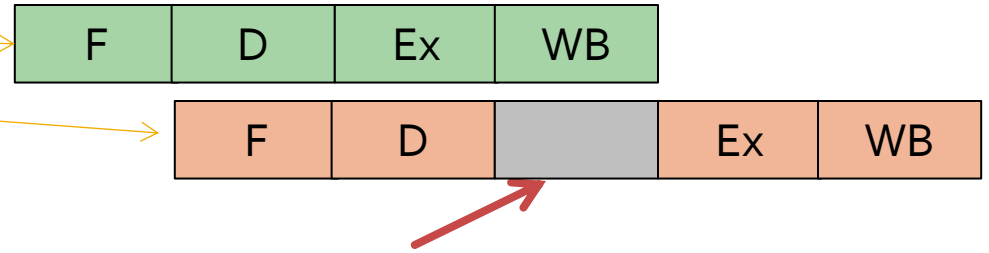
Program:

....

A=B*2

C=A+1

....



Stall (bubble)

Data dependencies

- Read after Write (RAW)

  ADD R1, --, --;

  SUB --, R1, --;

  - The second instruction must wait until the result is written into R1
  - The execution order of the instructions cannot be changed
- Write after Read (WAR) (anti-dependency)
  - ADD --, R1, --;
  - SUB R1, --, --;
  - The execution order of the instructions cannot be changed
- Write after Write (WAW)

  ADD R1, --, --;

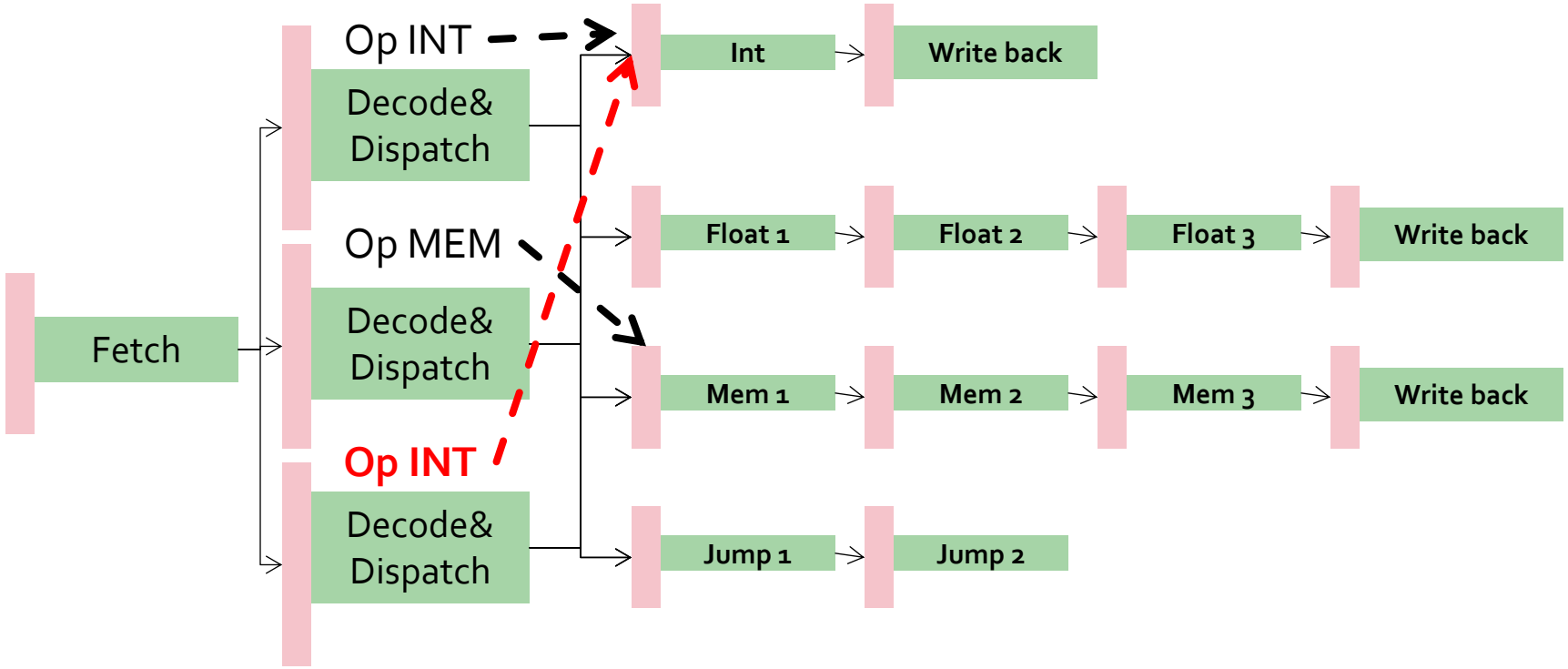
  SUB R1, --, --;

  - Problem: the second instruction executes faster than the first instruction
  - The write order must be preserved
  - The execution order of the instructions cannot be changed
- Read after Read (RAR) – causes no problems
  - ADD --, R1, --;
  - SUB --, R1, --;

Solutions to data dependences

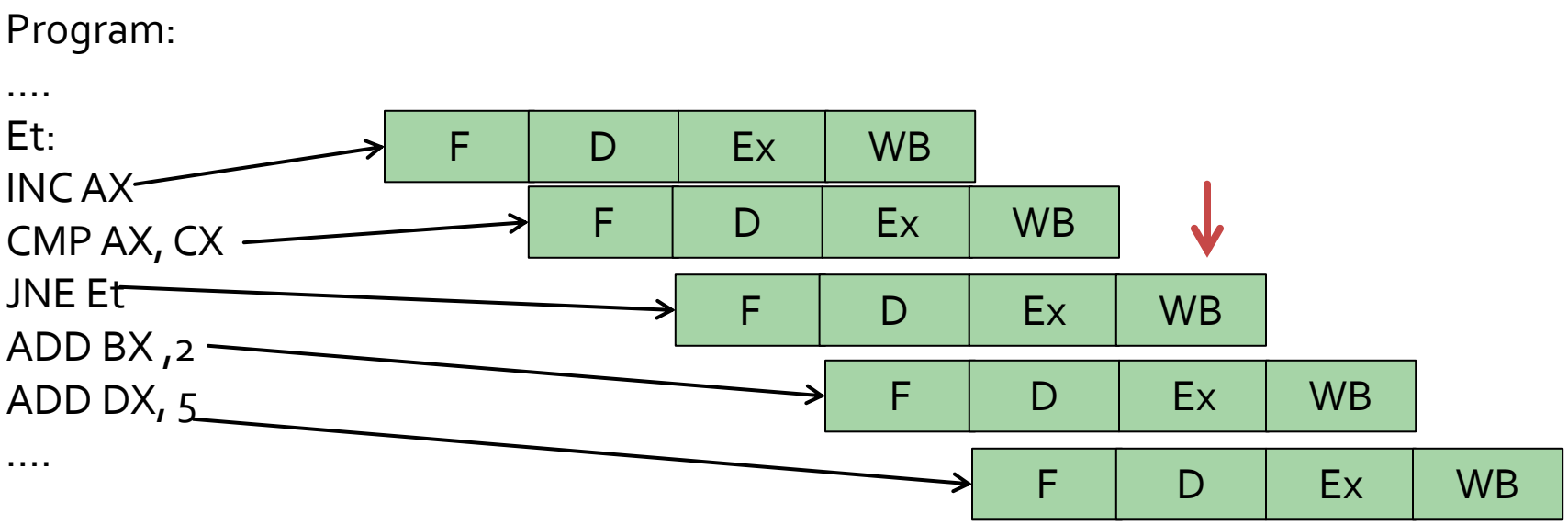
- Detection and waiting periods (stalls)
  - the instruction with the unresolved data dependency waits in the "instruction fetch" stage until the data is available
  - the following instructions are also stalled
- Register renaming
  - multiple copies of a register (see the alias registers of the Pentium Pro)
  - instructions with no logical dependency between them can get different copies of the same register
  - avoids the artificial data dependencies caused by the limited number of internal registers
- Forwarding – passes a result ahead, before it is written to its final destination (register or memory location)
- Out-of-order execution

Structural dependency



- Solutions: stalls, more identical functional units, o-o-o execution

Branches



- Solutions: stalls, branch prediction, speculative execution

Performance of pipelines

$$Speedup = \frac{CPI\ unpipelined}{CPI\ pipelined} = \frac{CPI\ unpipelined}{Ideal\ CPI + Pipeline\ stall\ cycles\ per\ instruction}$$

$$Speedup = \frac{Pipeline\ depth}{1 + Pipeline\ stall\ cycles\ per\ instruction}$$

$$Speedup = \frac{Pipeline\ depth}{1 + Branch\ frequency\ * Branch\ penalty}$$

Increasing execution performance

- Memory prefetching
- Out of order execution
- Branch prediction
- Speculative execution
- Speculative reads

Memory Prefetching

- Data is fetched into the cache **before** the program requests it, to hide memory latency
- **Hardware prefetchers** detect access patterns and fetch ahead automatically
- **Software prefetching:** the compiler or programmer inserts explicit hints (e.g., PREFETCHh on x86)

Out-of-Order Execution

- Instructions are executed as soon as their operands are ready, not in program order
- Hides latency: independent instructions proceed while others wait (e.g. on a cache miss)
- Structures that implement o-o-o: reservation stations, register renaming, reorder buffer (ROB)
- Results are **committed (retired) in program order**, so software sees sequential behavior

Branch Prediction

- The CPU guesses the outcome of a branch before the condition is known
- Keeps the pipeline full instead of stalling until the branch resolves
- **Direction prediction:** taken / not taken (pattern history, branch history)
- **Target prediction:** where indirect branches and returns go (BTB)
- Modern predictors are >95% accurate

Speculative Execution

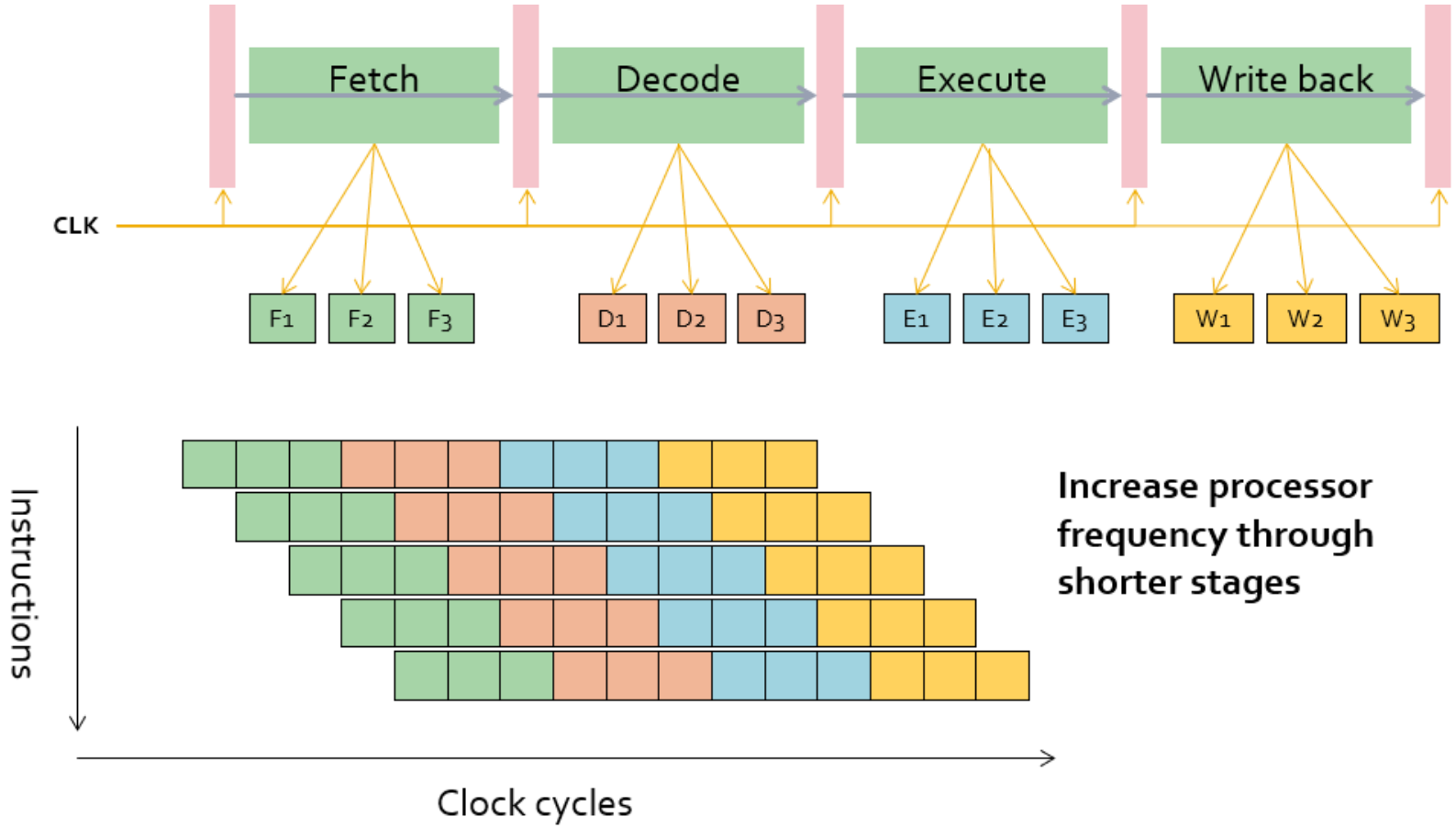
- Instructions are executed before it is known whether they are needed
- Driven by predictions: branch outcomes, branch targets, memory dependences
- Correct prediction: results are committed, and time is saved
- Misprediction: results are dropped and execution restarts on the correct path

Speculative Reads

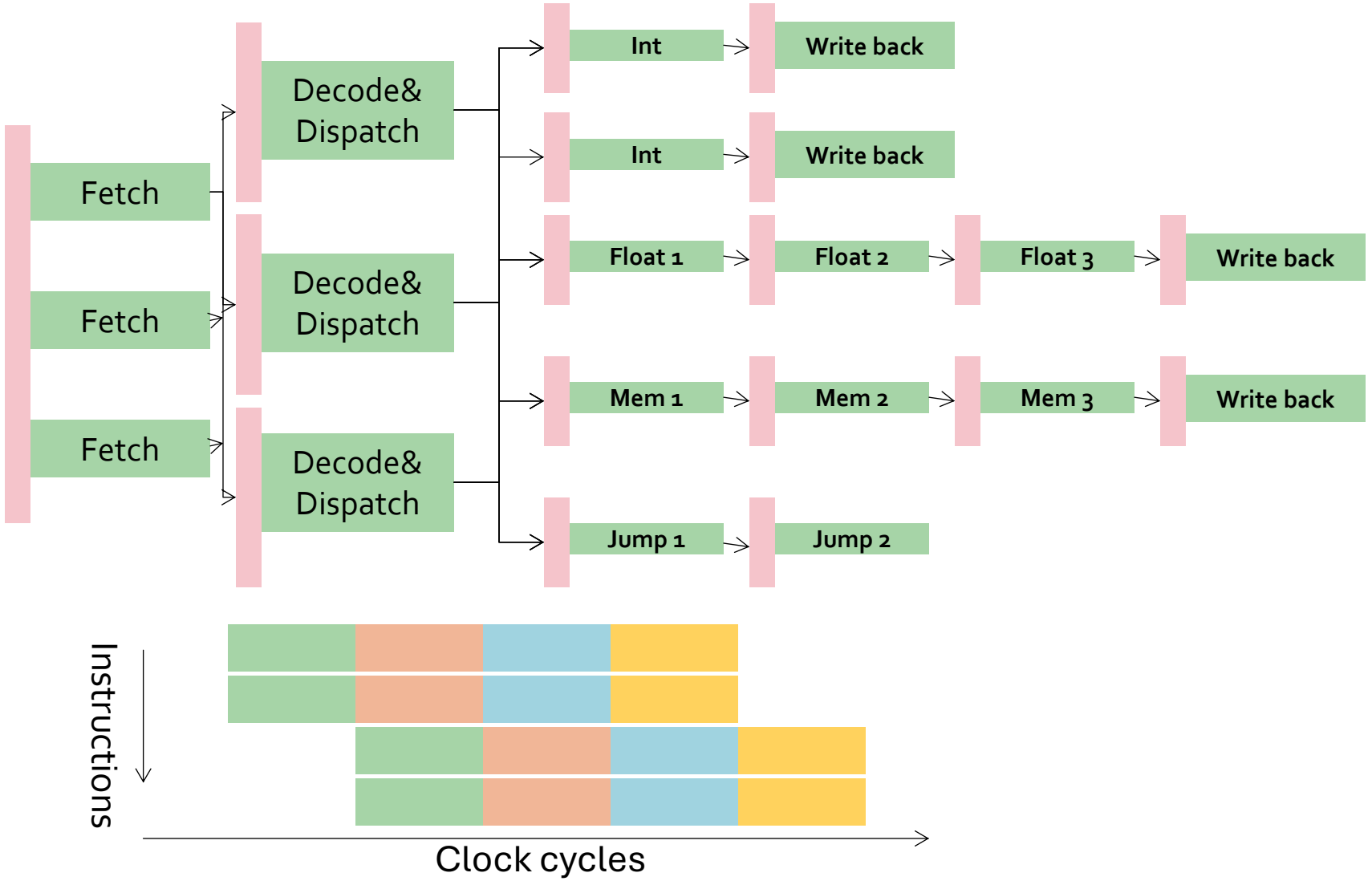
- Loads issued on a speculative path, before the branch or check that guards them has resolved
- May read data the program would never access architecturally (out-of-bounds, other privilege levels)
- The value can be forwarded to dependent instructions before the speculation is resolved

More...

Superpipeline



Superscalar (CPI<1)

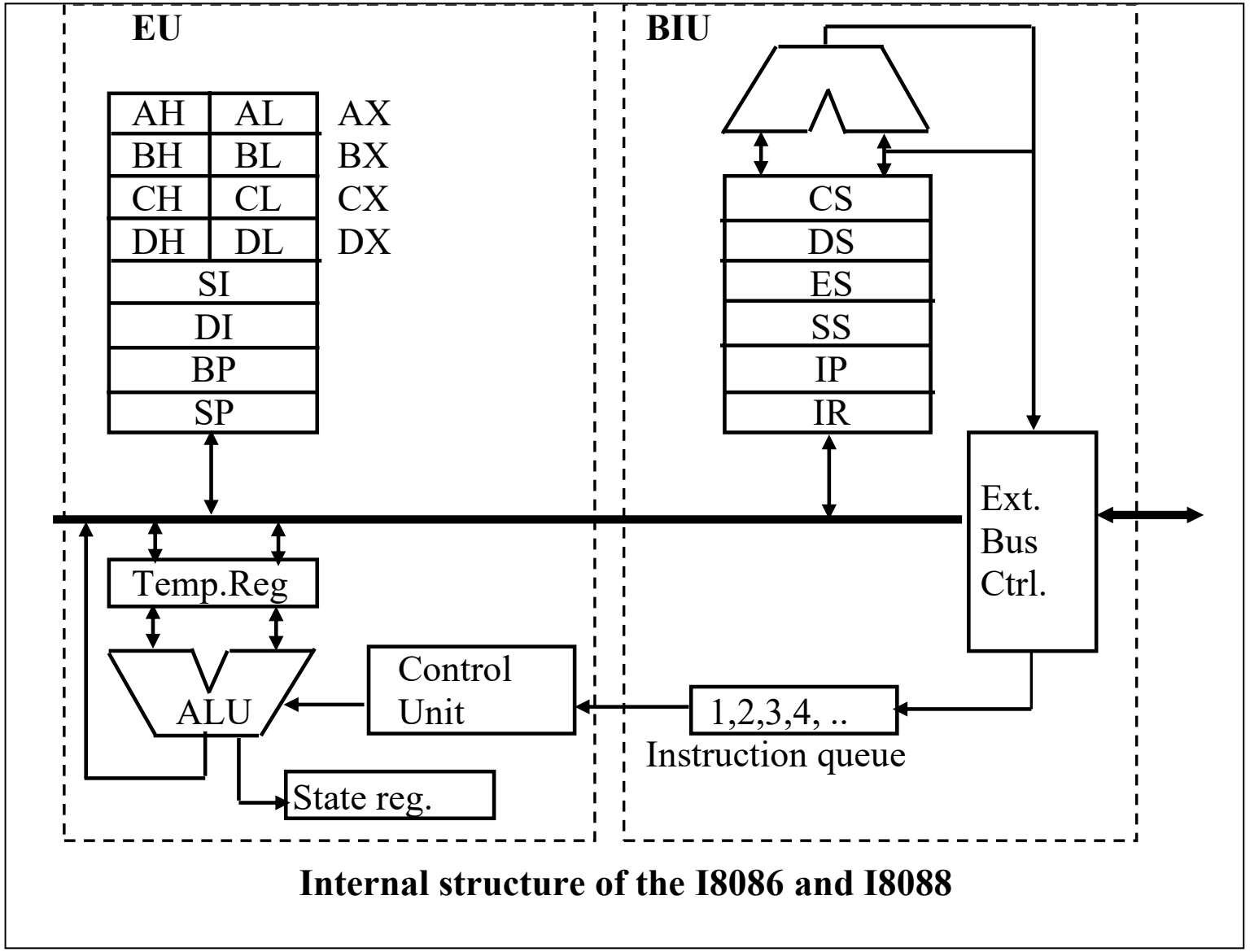


Intel architectures history

Complex, backward-compatible

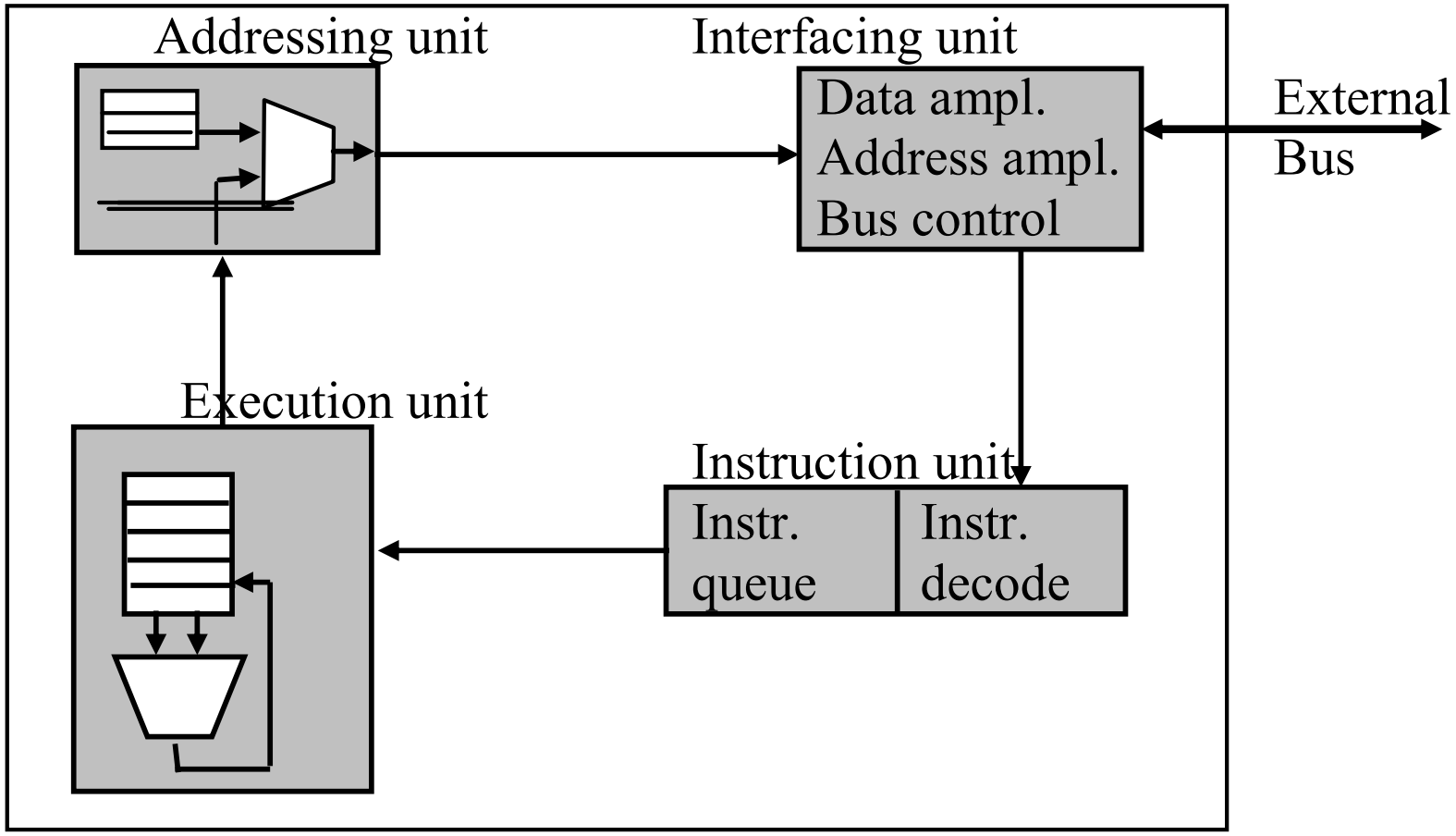
- **16-bit Architecture (1978):** 8086, 8088
  - 20-bit addressing, 16-bit data bus
  - Segmented memory (1MB, segments of 64KB), 16-bit segment registers + 16-bit pointer for offset
  - Operates in real mode
- **Introduction of Protected Mode** (1982): Intel 286
  - 24-bit addressing, 16MB segmented memory
  - Operating modes: protected, real, virtual
- **32-bit Architecture IA-32 (1985):** Intel 386
  - 32-bit addressing and data
  - 4GB memory with segmentation and paging
  - Compatibility with 16-bit architecture
  - Operating modes: protected, real, virtual
  - Pipelining
- **Intel 486** (1989): Pipeline improvements, Integrated x87 FPU
- **Pentium** (1993): Branch prediction, MMX technology
- **P6 Family** (1995–1999): Superscalar microarchitecture, SSE technology, Multiprocessor support
- **Pentium 4 (**2000–2006): NetBurst microarchitecture, Hyper-Threading technology, Virtualization support
- **2005**: Intel 64 architecture, Intel Core microarchitecture, Multi-Core technology, Smart Cache, Dynamic Power Coordination, etc.
- **2019**: Intel Core 10th Generation

I8086



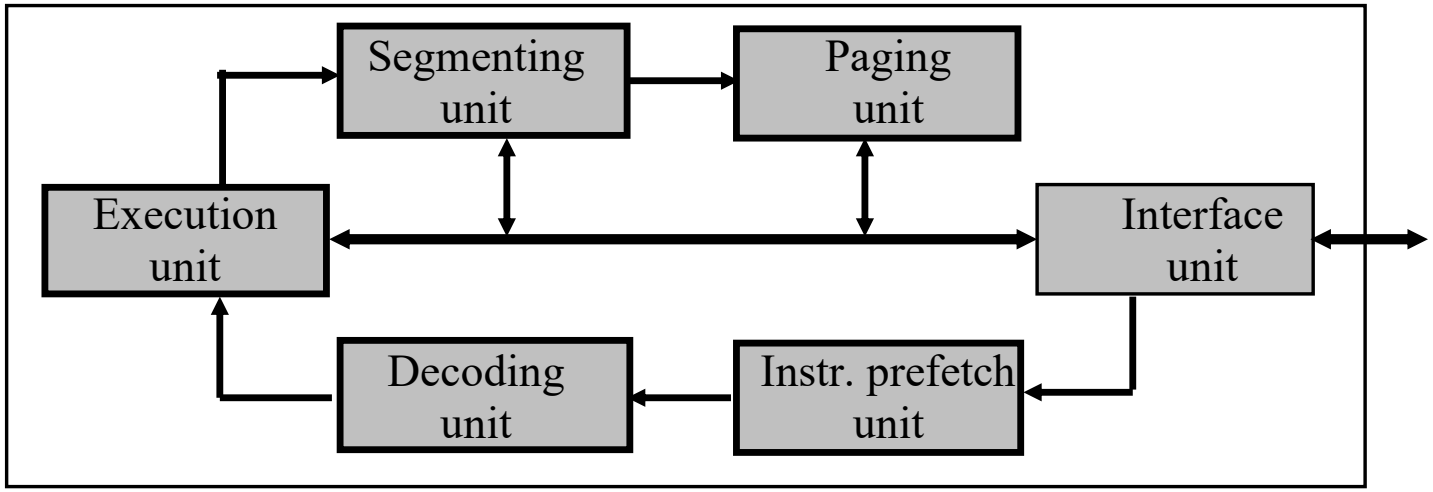
Internal structure of the I8086 and I8088

I80286

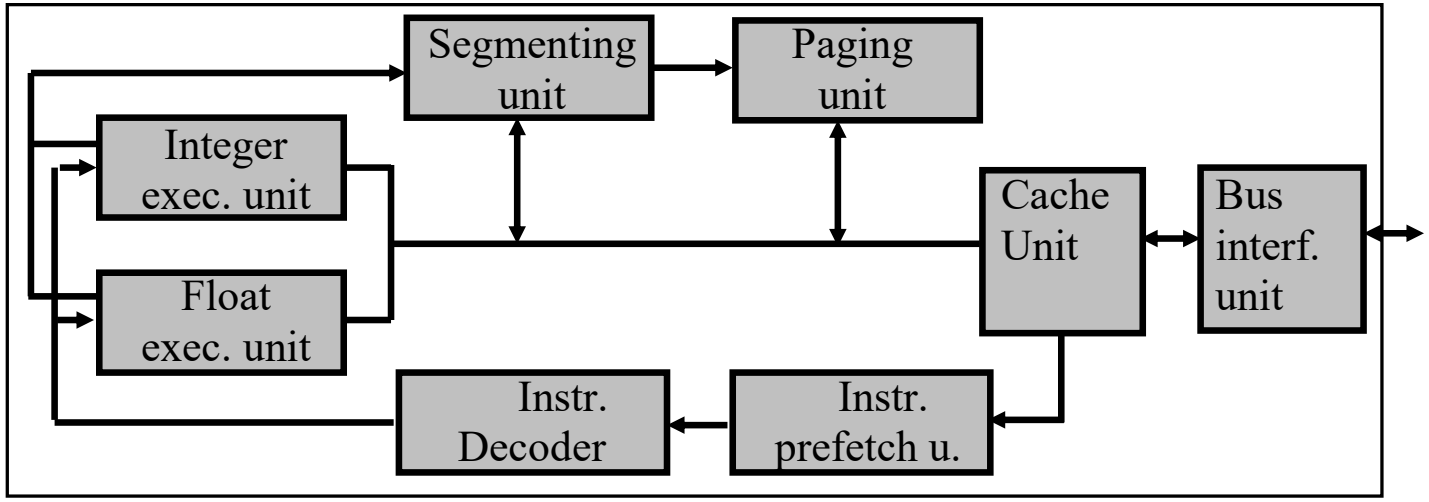


Internal structure of the I80286 processor

I80386 and I80486



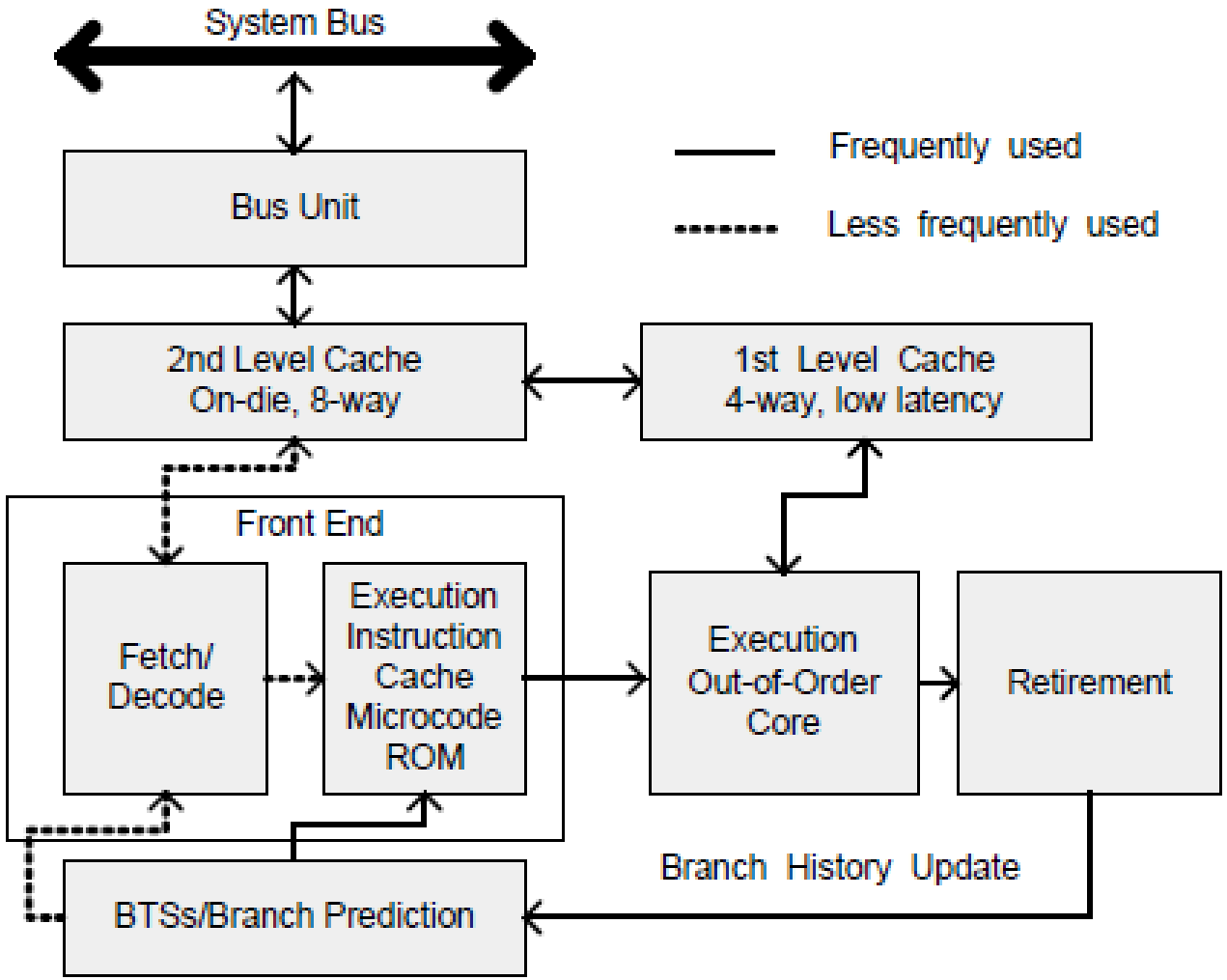
Internal structure of the I80386 processor



Internal structure of the I80486

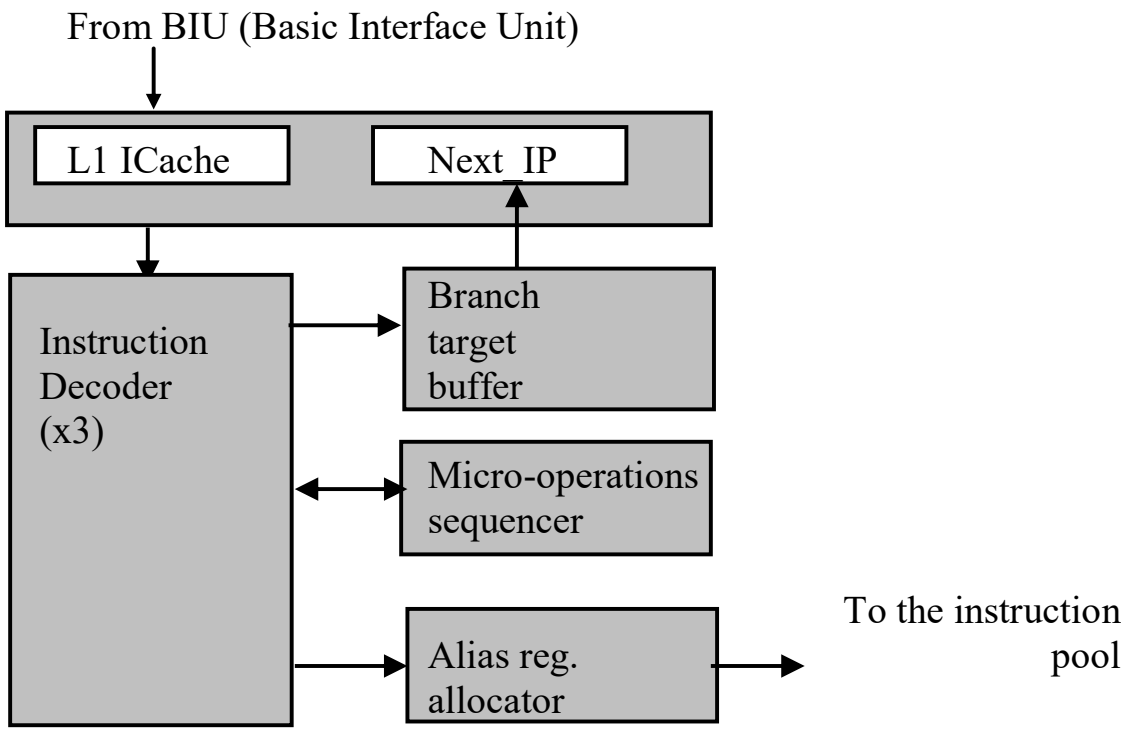
Pentium Pro - P6 superscalar architecture

- 3 autonomous units
- 12 stage pipeline
- Tomasulo implementation for hazards solution



Instruction fetch and decode unit

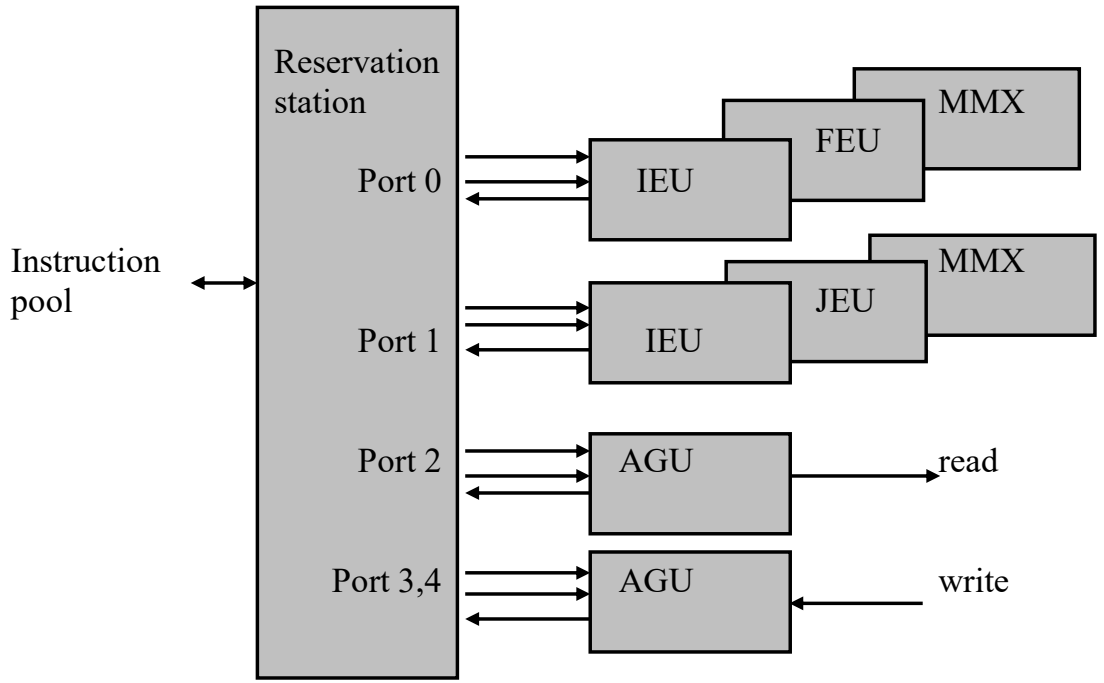
- Fetches and decodes instructions in advance
- In-order unit
- 3 instructions decoded / clock
- Branch prediction
- Components:
  - Decoders (3 units)
  - Address generator unit (next_IP)
  - Branch target buffer - BTB
  - Micro-operation sequencer
  - Alias registers allocator



Instruction fetch and decoding unit

Instruction dispatch and execution unit

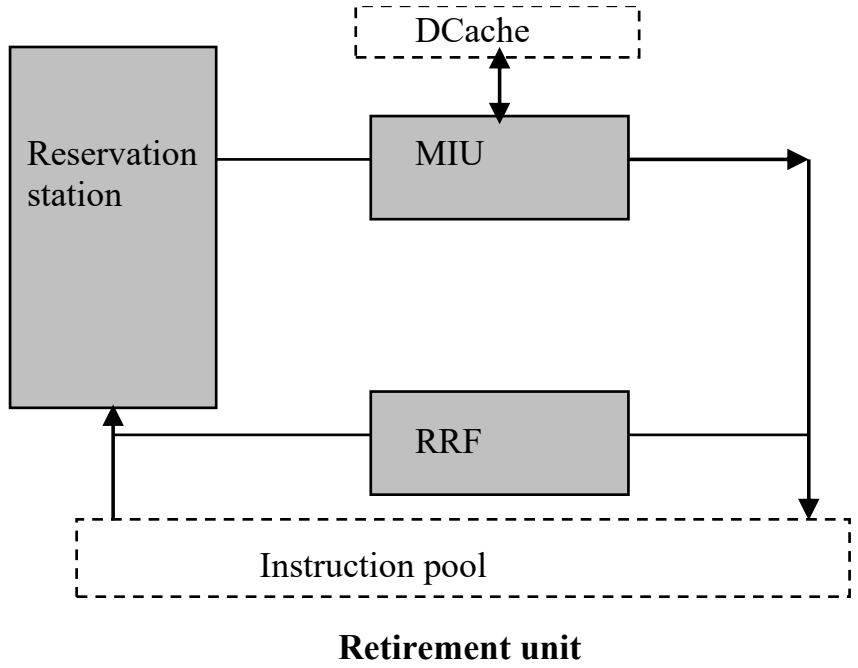
- Responsible for executing the instructions
- Out-of-order unit
- 7 execution units + reservation station
  - IEU – Integer Execution Unit
  - FEU – Floating-point Execution Unit
  - MMX – Multimedia execution unit
  - AGU – Address generation unit
  - JGU – Jump generation unit



Instruction dispatch and execute

Retirement (commit) unit

- Restores the original program order of the instructions (of the results)
- In-order unit
- Components:
  - MIU – memory interface unit
  - RRF – Retirement register file



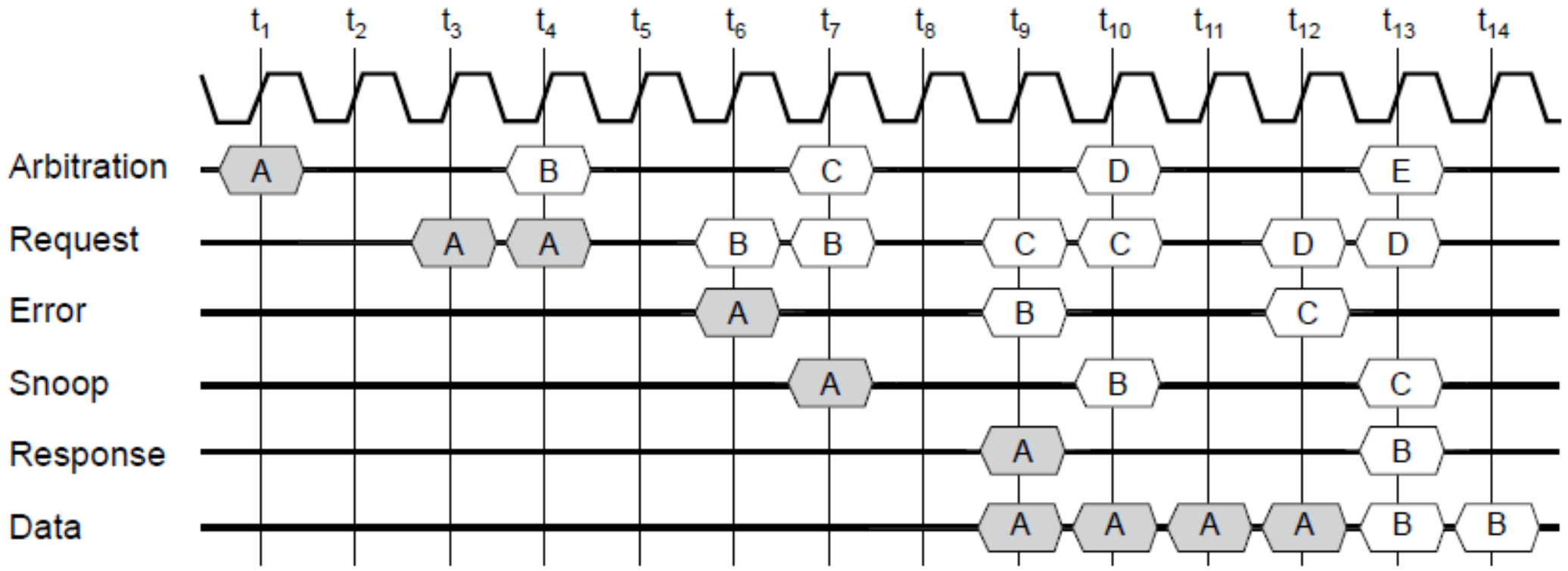
Hazard resolution in the P6 architecture

- Control hazards:
  - complex branch prediction, BTB, next-address predictor
  - Speculative execution of instructions – out-of-order
  - Executes instructions from both branches of an “if”
- Data hazards:
  - Copy (alias) registers: register renaming and more internal registers (40) than those visible to the programmer
  - Speculative execution of instructions – out-of-order
  - Building dependency trees
- Structural hazards
  - Multiple execution units (7 ALUs)
  - Separate data and instruction caches
  - Reservation station
- Essentially an implementation of Tomasulo's algorithm

The P6 bus

- Main features of the P6 bus:
  - the bus operates synchronously; every signal is sampled on the clock edge
  - transfers are done through transactions that can be executed in parallel
  - it is a multiprocessor bus; several processors on the same bus
  - block transfers are preferred
  - there are error detection and correction mechanisms
  - there are mechanisms that ensure cache coherence
  - a new digital technology (special buffers) that enables high-frequency transmission on the bus

P6 bus timing diagram



Gwennap, Linley. "Intel's P6 bus designed for multiprocessing." Microprocessor Report 9.7 (1995): 1.

Transfers on the P6 bus

- Parallel (pipelined) transactions
- Phases:
  - Arbitration – decides which master unit gets access to the bus
  - Request – specifies the transfer (read or write, start address, number of bytes)
  - Error – detection and handling of transmission errors (ECC – error-correcting code on the data, and parity on the address and command signals)
  - Snoop – detection and resolution of cache coherence problems
  - Response – specifies the type of response (immediate, deferred, retry)
  - Transfer – data transfer according to the request

Intel Core microarchitecture

- **Front End**
  - Instruction pre-fetch
  - Decodes 4-5 instr. per cycle
  - Branch prediction – various conditional instr. types
- **Execution Core**
  - Up to 6 micro-ops per cycle
  - Up to 4 instr. retired per cycle
  - 3xALU, 3xSIMD
  - 8 FP operations per cycle
  - High-latency operations – hardware pipeline

